In [1]:
```python
# Core Data Analysis & Manipulation
import pandas as pd
import numpy as np

# Database Connection (jaise SQLite)
import sqlite3

# Data Visualization (Graphs & Charts)
import matplotlib.pyplot as plt
import seaborn as sns
```

In [ ]:
```python
# 1. Import Database / Data
```

In [2]:
```python
conn = sqlite3.connect('customer_churn.db')

sql_query = """
    SELECT name
    FROM sqlite_master
    WHERE type='table';
"""

tables = pd.read_sql(sql_query, conn)

# create dataframe for each table
for table_name in tables['name']:
    df = pd.read_sql(f"SELECT * FROM {table_name}", conn)
    globals()[f"df_{table_name}"] = df
    print(f"Created dataframe: df_{table_name}")

conn.close()
```

```
Created dataframe: df_db_customer
Created dataframe: df_db_subscription
Created dataframe: df_db_support
```

In [3]:
```python
# Print table names and column names
conn = sqlite3.connect('customer_churn.db')

for table_name in tables['name']:
    print(f"\nTable Name: {table_name}")
    # Get column information
    columns_query = f"PRAGMA table_info({table_name});"
    columns = pd.read_sql(columns_query, conn)
    print("Columns:")
    print(columns['name'].tolist())

# Close connection
conn.close()
```

```
Table Name: db_customer
Columns:
['customerid', 'name', 'country', 'state', 'gender', 'dob', 'interests', 'pincod
e']

Table Name: db_subscription
Columns:
['customerid', 'subscription_start_date', 'subscription_type', 'renewal_date', 'p
lan_type', 'contract_type', 'cancellation_date', 'cancellation_reason', 'monthly_
charges', 'cltv', 'churn_score']

Table Name: db_support
Columns:
['customerid', 'complaint_date', 'escalations', 'csat_score', 'col_1', 'comment']
```

In [ ]:
```
# 2. Data Cleaning
```

In [8]:
```
#df_db_customer.head()
df_db_customer.tail()
```

Out[8]:

	customerid	name	country	state	gender	dob	interests	pincode
16	0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19 00:00:00	NaN	None
17	0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02 00:00:00	NaN	None
18	0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30 00:00:00	NaN	None
19	0023-HGHWL	rishabh	India	Uttar Pradesh	Men	1991-05-14 00:00:00	NaN	None
20	0023-UYUPN	sudevi	India	Maharashtra	Women	1977-10-06 00:00:00	NaN	None

In [5]:
```
df_db_customer.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 8 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   customerid  21 non-null     str
 1   name        21 non-null     str
 2   country     18 non-null     str
 3   state       21 non-null     str
 4   gender      21 non-null     str
 5   dob         21 non-null     str
 6   interests   4 non-null      str
 7   pincode     0 non-null      object
dtypes: object(1), str(7)
memory usage: 1.4+ KB
```

In [ ]:
```
# a. Renmae columns - name
# b. Drop columns - interest and pincode
# c. Change data type - dob
# d. Data standardization - gender
# e. Fix missing value - country
```

In [9]:
```
# a. Renmae columns - name

df_db_customer.rename(columns = {'name' : 'customer_name'}, inplace= True)
```

Out[9]:

	customerid	customer_name	country	state	gender	dob	interests	pin
0	0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12 00:00:00	travel	N
1	0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23 00:00:00	NaN	N
2	0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15 00:00:00	movie	N
3	0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30 00:00:00	NaN	N
4	0013-EXCHZ	mira	India	Delhi	Female	1990-05-05 00:00:00	drama	N
5	0013-MHZWF	durga	NaN	Delhi	Women	1988-12-10 00:00:00	NaN	N
6	0013-SMEOE	mina	India	Meghalaya	Female	1976-09-21 00:00:00	NaN	N
7	0014-BMAQU	madan	India	Rajasthan	Male	1999-03-14 00:00:00	NaN	N
8	0015-UOCOJ	maya	NaN	Kathmandu	Women	1985-07-07 00:00:00	NaN	N
9	0016-QLJIS	arjun	Nepal	Kathmandu	Male	1993-10-29 00:00:00	NaN	N
10	0017-DINOC	shiva	India	Maharashtra	Men	1997-01-22 00:00:00	NaN	N
11	0017-IUDMW	rangadevi	India	Karnataka	Female	1981-06-18 00:00:00	NaN	N
12	0018-NYROU	chitra	NaN	Telangana	Female	2004-12-01 00:00:00	NaN	N
13	0019-EFAEP	raju	India	Meghalaya	Female	1992-04-25 00:00:00	NaN	N
14	0019-GFNTW	Madhav	India	Uttar Pradesh	Men	1979-11-11 00:00:00	NaN	N

	customerid	customer_name	country	state	gender	dob	interests	pin
15	0020-INWCK	parvati	India	Delhi	Female	1986-02-28 00:00:00	job	N
16	0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19 00:00:00	NaN	N
17	0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02 00:00:00	NaN	N
18	0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30 00:00:00	NaN	N
19	0023-HGHWL	rishabh	India	Uttar Pradesh	Men	1991-05-14 00:00:00	NaN	N
20	0023-UYUPN	sudevi	India	Maharashtra	Women	1977-10-06 00:00:00	NaN	N

In [14]:
```
# b. Drop columns - interest and pincode

#df_db_customer.drop(df_db_customer.columns[-2:], axis=1)
#df_db_customer.drop(df_db_customer.columns[6:], axis=1)

df_db_customer.drop(columns=['interests', 'pincode'], inplace=True)
```

In [19]:
```
df_db_customer.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   customerid     21 non-null     str
 1   customer_name  21 non-null     str
 2   country        18 non-null     str
 3   state          21 non-null     str
 4   gender         21 non-null     str
 5   dob            21 non-null     datetime64[us]
dtypes: datetime64[us](1), str(5)
memory usage: 1.1 KB
```

In [18]:
```
# c. Change data type - dob

df_db_customer['dob'] = pd.to_datetime(df_db_customer['dob'])
```

In [21]:
```
# d. Data standardization - gender

df_db_customer['gender'] = df_db_customer['gender'].replace({'Men' : 'Male', 'Wo
```

In [23]:
```
# e. Fix missing value - country

df_db_customer[df_db_customer['country'].isna()]
```

Out[23]:

	customerid	customer_name	country	state	gender	dob
5	0013-MHZWF	durga	NaN	Delhi	Female	1988-12-10
8	0015-UOCOJ	maya	NaN	Kathmandu	Female	1985-07-07
12	0018-NYROU	chitra	NaN	Telangana	Female	2004-12-01

In [28]:
```
# country and state - unique value pair

state_country_mapping = df_db_customer.dropna(subset=['country']).set_index('sta

df_db_customer['country'] = df_db_customer['country'].fillna(df_db_customer['sta
```

In [29]:
```
df_db_customer[df_db_customer['country'].isna()]
```

Out[29]:

	customerid	customer_name	country	state	gender	dob

In [30]:
```
df_db_customer[['country', 'state']]
```

Out[30]:

	country	state
0	India	Maharashtra
1	India	Karnataka
2	India	Delhi
3	India	Nagaland
4	India	Delhi
5	India	Delhi
6	India	Meghalaya
7	India	Rajasthan
8	Nepal	Kathmandu
9	Nepal	Kathmandu
10	India	Maharashtra
11	India	Karnataka
12	India	Telangana
13	India	Meghalaya
14	India	Uttar Pradesh
15	India	Delhi
16	India	Meghalaya
17	India	Rajasthan
18	India	Telangana
19	India	Uttar Pradesh
20	India	Maharashtra

In [31]:
```
df_db_subscription.head()
```

Out[31]:

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	cont
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	
2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	
3	0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	
4	0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	

In [32]:
```
df_db_subscription.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   customerid               21 non-null     str
 1   subscription_start_date  21 non-null     str
 2   subscription_type        21 non-null     str
 3   renewal_date             21 non-null     str
 4   plan_type                21 non-null     str
 5   contract_type            21 non-null     str
 6   cancellation_date        6 non-null      str
 7   cancellation_reason      6 non-null      str
 8   monthly_charges          21 non-null     float64
 9   cltv                     21 non-null     int64
 10  churn_score              21 non-null     int64
dtypes: float64(1), int64(2), str(8)
memory usage: 1.9 KB
```

In [36]:
```
date_col = ['subscription_start_date', 'renewal_date', 'cancellation_date']

df_db_subscription[date_col] = df_db_subscription[date_col].apply(pd.to_datetime
df_db_subscription.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   customerid               21 non-null     str
 1   subscription_start_date  21 non-null     datetime64[us]
 2   subscription_type        21 non-null     str
 3   renewal_date             21 non-null     datetime64[us]
 4   plan_type                21 non-null     str
 5   contract_type            21 non-null     str
 6   cancellation_date        6 non-null      datetime64[us]
 7   cancellation_reason      6 non-null      str
 8   monthly_charges          21 non-null     float64
 9   cltv                     21 non-null     int64
 10  churn_score              21 non-null     int64
dtypes: datetime64[us](3), float64(1), int64(2), str(5)
memory usage: 1.9 KB
```

In [37]:
```
df_db_support.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   customerid      9 non-null      str
 1   complaint_date  9 non-null      str
 2   escalations     9 non-null      str
 3   csat_score      9 non-null      int64
 4   col_1           0 non-null      object
 5   comment         4 non-null      str
dtypes: int64(1), object(1), str(4)
memory usage: 564.0+ bytes
```

In [38]:
```
df_db_support.drop(columns=['col_1', 'comment'], inplace=True)
```

In [40]:
```
df_db_support['complaint_date'] = pd.to_datetime(df_db_support['complaint_date']

df_db_support.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 4 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   customerid      9 non-null      str
 1   complaint_date  9 non-null      datetime64[us]
 2   escalations     9 non-null      str
 3   csat_score      9 non-null      int64
dtypes: datetime64[us](1), int64(1), str(2)
memory usage: 420.0 bytes
```

In [ ]:
```
```

In [ ]:
```
# 3. Feature Engineering and Data Analysis
```

In [93]:
```
# Create a new col using existing col - churn flag
df_db_subscription['churn_flag'] = np.where(df_db_subscription['cancellation_dat
```

In [94]:
```
df_db_subscription.head()
```

Out[94]:

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	cont
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	
2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	
3	0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	
4	0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	

In [47]:
```
# First fix support table duplicates then merge
df = (df_db_subscription
              .merge(df_db_customer, on = 'customerid', how= 'left')
              .merge(df_db_support, on = 'customerid', how= 'left'))
```

In [49]:
```
df_db_subscription.shape
```

Out[49]:
```
(21, 12)
```

In [48]:
```
df.shape
```

Out[48]:
```
(23, 20)
```

In [51]:
```
df_db_subscription['customerid'].nunique()
```

Out[51]:
```
21
```

In [52]:
```
df_db_support['customerid'].nunique()
```

Out[52]: 7

In [53]:
```
df_db_support['customerid'].size
```

Out[53]: 9

In [59]:
```
df_db_support
```

Out[59]:

	customerid	complaint_date	escalations	csat_score	complaint_count
0	0003-MKNFE	2024-08-28	N	60	2
1	0003-MKNFE	2024-08-28	Y	10	2
2	0013-EXCHZ	2024-01-20	Y	20	1
3	0013-MHZWF	2025-03-18	N	90	1
4	0013-SMEOE	2024-11-01	N	30	1
5	0017-IUDMW	2024-04-10	Y	25	1
6	0019-EFAEP	2024-09-27	Y	30	1
7	0022-TCJCI	2024-09-13	Y	10	2
8	0022-TCJCI	2024-09-14	N	90	2

In [58]:
```
df_db_support['complaint_count'] = df_db_support.groupby('customerid')['customer
```

In [61]:
```
df_db_support = df_db_support.sort_values('complaint_date').drop_duplicates('cus
```

In [62]:
```
df_db_support['customerid'].size
```

Out[62]: 7

In [96]:
```
# merge df
df = (df_db_subscription
                .merge(df_db_customer, on = 'customerid', how= 'left')
                .merge(df_db_support, on = 'customerid', how= 'left'))
```

In [64]:
```
df.shape
```

Out[64]: (21, 21)

In [66]:
```
df.to_csv('exported_churn_data.csv', index=False)
```

In [ ]:
```

```

In [ ]:
```
# Data Analysis
```

In [ ]:
```
# 1. Churn Rate
```

In [67]:
```
df.columns
```

Out[67]:
```
Index(['customerid', 'subscription_start_date', 'subscription_type',
       'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
       'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
       'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
       'complaint_date', 'escalations', 'csat_score', 'complaint_count'],
      dtype='str')
```

In [71]:
```
churn_rate = df['churn_flag'].mean()*100
print("Churn Rate = ", round(churn_rate,2), "%")
```

```
Churn Rate =  28.57 %
```

In [72]:
```
# 2. Retention Rate
retention_rate = 100 - churn_rate
print("retention_rate = ", round(retention_rate,2), "%")
```

```
retention_rate =  71.43 %
```

In [78]:
```
# 3. Churn by Plan type
churn_by_plan = (df.groupby('plan_type')['churn_flag'].mean().mul(100).round(2).
print("Churn by plan = ", churn_by_plan)
```

```
Churn by plan =    plan_type  churn_rate_pct
0     Basic           60.00
1   Premium           14.29
2  Standard           22.22
```

In [82]:
```
# 4. ARPU - Avg Revenue per user
arpu = df['monthly_charges'].mean()
print('ARPU = ',round(arpu,2))
```

```
ARPU =  18.85
```

In [ ]:
```
# homework - calculate customer age
```

In [89]:
```
# 5. Avg Customer Tenure
# count of days users has used our service : cancellation date else current rate
today = pd.Timestamp.today()

df['tenure_days'] = np.where(
           df['cancellation_date'].notna(),

    (df['cancellation_date'] - df['subscription_start_date']).dt.days,

    (today - df['subscription_start_date']).dt.days
)

avg_tenure = df['tenure_days'].mean()
print("Avg Tenure (Days) = ", round(avg_tenure),0)
```

```
Avg Tenure (Days) =  1560 0
```

In [100…
```
# 6. Revenue at risk - revenue last from churned users
revenue_at_risk = df.loc[df['churn_flag']==1, 'monthly_charges'].sum()
print("Revenue at Risk (Rs 'K') =", revenue_at_risk)
```

```
Revenue at Risk (Rs 'K') = 73.94
```

In [104…
```
# 7. Escalation Rate
escalation_rate = (df['escalations']=='Y').mean()*100
print("Escalation Rate = ", round(escalation_rate, 2), "%")
```

```
Escalation Rate =  19.05 %
```

In [107…
```python
# 8. Avg Complaint Per User
avg_complaints = df['complaint_count'].sum() / df['customerid'].nunique()
print("Avg Complaints Per User = ", round(avg_complaints,2))
```

```
Avg Complaints Per User =  0.43
```

In [124…
```python
# 9. Correlation: Escalation vs Churn

df['escalation'] = np.where(df['escalations'] == 'Y', 1, 0)
corr_df = df[['escalation', 'churn_flag']].dropna()

correlation = corr_df['escalation'].corr(corr_df['churn_flag'])
print("Correlation between escalation vs churn is =", round(correlation, 2))
```

```
Correlation between escalation vs churn is = 0.77
```

In [126…
```python
# 10. Create a column using existing column - Churn Risk

conditions = [
    (df['churn_score'] < 50),
    (df['churn_score'] >= 50) & (df['churn_score'] < 70),
    (df['churn_score'] >= 70)
]

choices = ['low', 'med', 'high']

df['churn_risk'] = np.select(
    conditions,
    choices,
    default='unknown'
)
```

In [ ]:
```python
# 4. Visualization using Matplotlib
```

In [132…
```python
df_visual = df.copy()
```

In [136…
```python
df_visual.columns
```

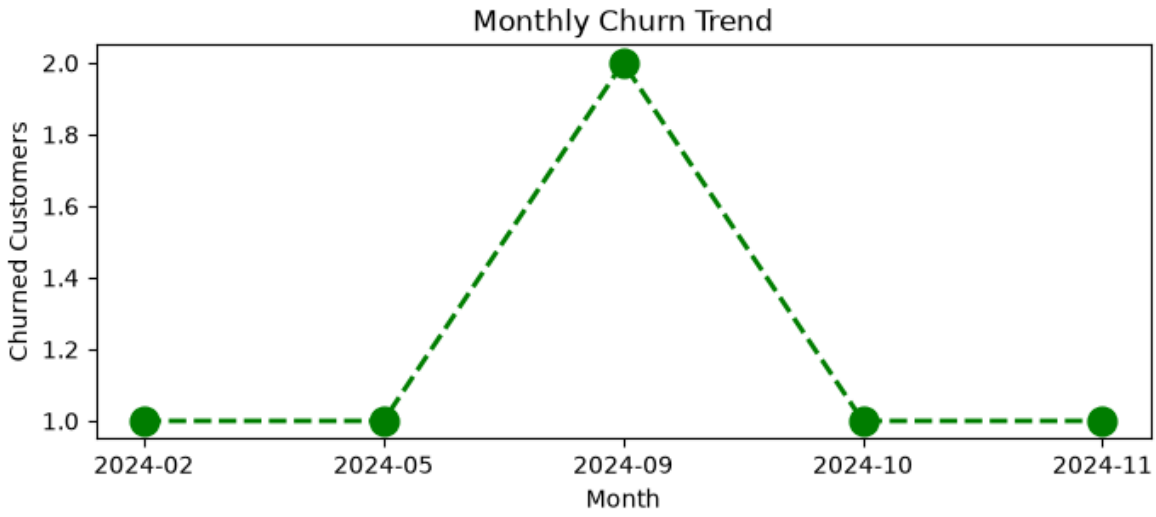
Out[136…
```
Index(['customerid', 'subscription_start_date', 'subscription_type',
       'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
       'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
       'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
       'complaint_date', 'escalations', 'csat_score', 'complaint_count',
       'escalation', 'churn_risk'],
      dtype='str')
```

In [143…
```python
# 4.1 Monthly Churn Trend (Time Series KPI)

df_visual['cancellation_month'] = df_visual['cancellation_date'].dt.to_period('M

churn_trend = df_visual[df_visual['churn_flag'] == 1].groupby('cancellation_mont
plt.figure(figsize=(8,3))
plt.plot(churn_trend.index.astype(str), churn_trend.values, color='green',marker

plt.title('Monthly Churn Trend')
plt.xlabel('Month')
plt.ylabel('Churned Customers')
plt.show()
```



In [158…

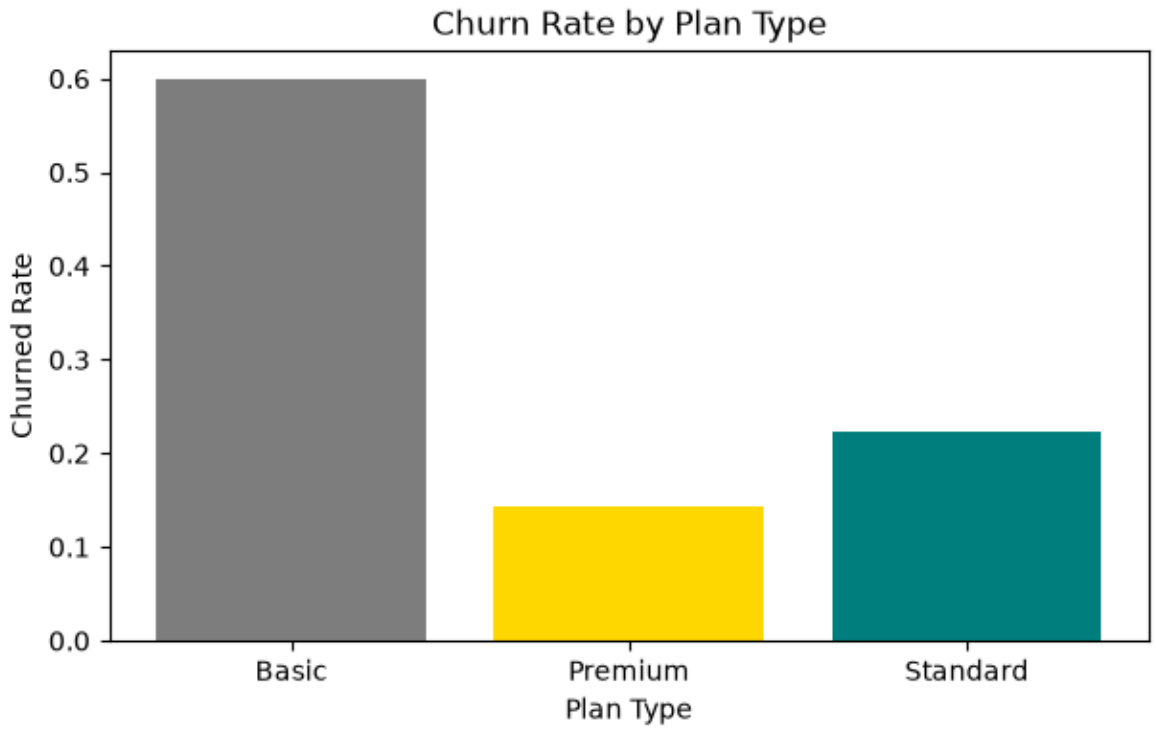
```
# 4.2 Churn by Plan type
churn_plan = df_visual.groupby('plan_type')['churn_flag'].mean()

colors = ['grey', 'gold', 'teal']

plt.figure(figsize=(7,4))

plt.bar(churn_plan.index, churn_plan.values, color = colors)

plt.title('Churn Rate by Plan Type')
plt.xlabel('Plan Type')
plt.ylabel('Churned Rate')
plt.show()
```



In [159…

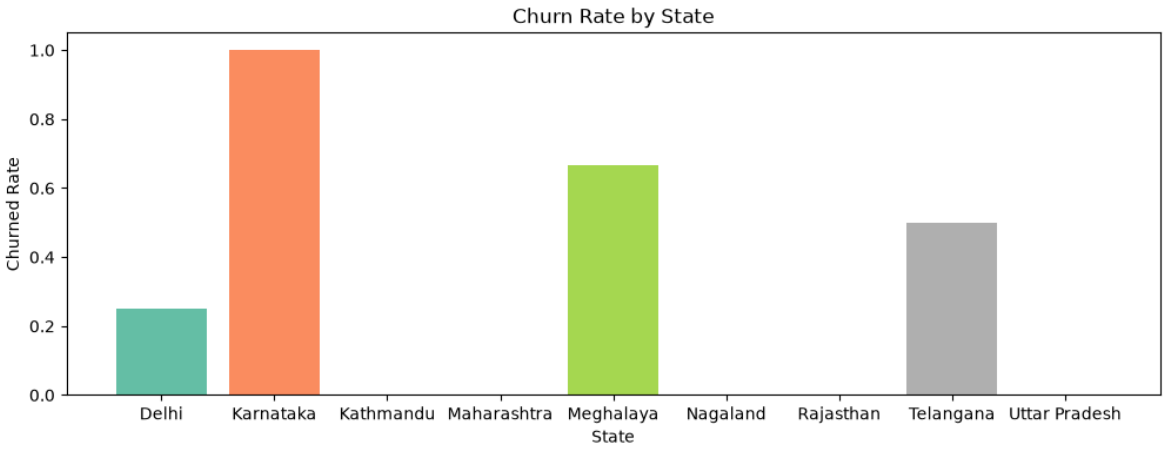
```
# 4.3 Churn by States
churn_plan = df_visual.groupby('state')['churn_flag'].mean()

#colors = ['grey', 'gold', 'teal']
colors = plt.cm.Set2(np.linspace(0, 1, len(churn_plan)))
```

```
plt.figure(figsize=(12,4))

plt.bar(churn_plan.index, churn_plan.values, color = colors)

plt.title('Churn Rate by State')
plt.xlabel('State')
plt.ylabel('Churned Rate')
plt.show()
```



In [160…

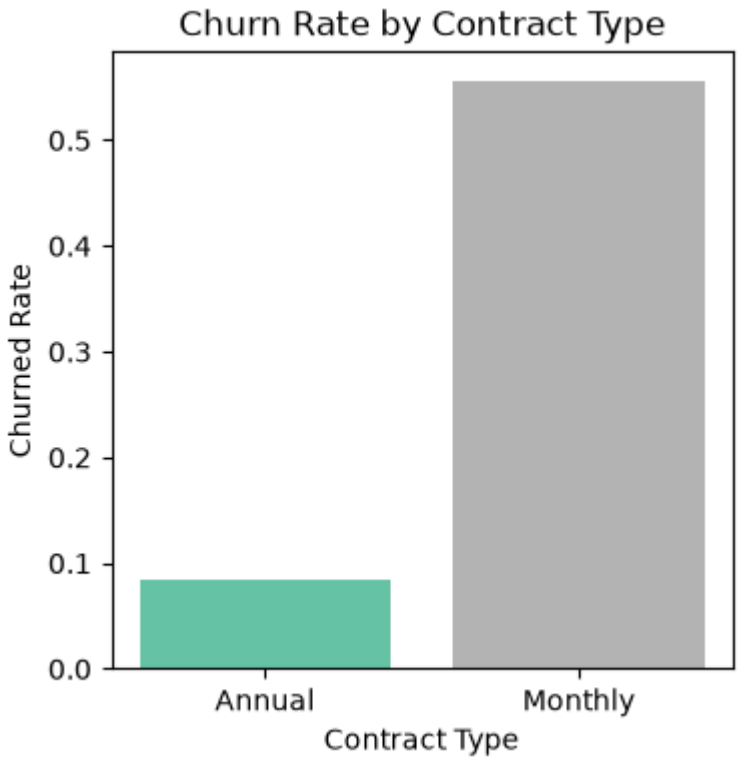
```
# 4.4 Churn by Contract Type
churn_plan = df_visual.groupby('contract_type')['churn_flag'].mean()

#colors = ['grey', 'gold', 'teal']
colors = plt.cm.Set2(np.linspace(0, 1, len(churn_plan)))

plt.figure(figsize=(4,4))

plt.bar(churn_plan.index, churn_plan.values, color = colors)

plt.title('Churn Rate by Contract Type')
plt.xlabel('Contract Type')
plt.ylabel('Churned Rate')
plt.show()
```



In [161…

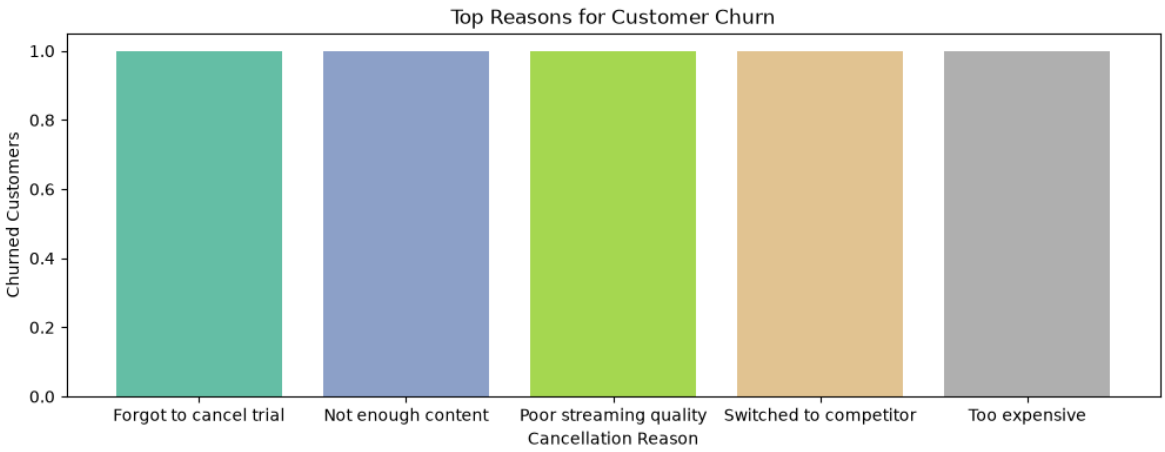
```
# 4.5 Churn by Cancellation Reason
churn_plan = df_visual.groupby('cancellation_reason')['churn_flag'].mean()

#colors = ['grey', 'gold', 'teal']
colors = plt.cm.Set2(np.linspace(0, 1, len(churn_plan)))

plt.figure(figsize=(12,4))

plt.bar(churn_plan.index, churn_plan.values, color = colors)

plt.title('Top Reasons for Customer Churn')
plt.xlabel('Cancellation Reason')
plt.ylabel('Churned Customers')
plt.show()
```



In [165…

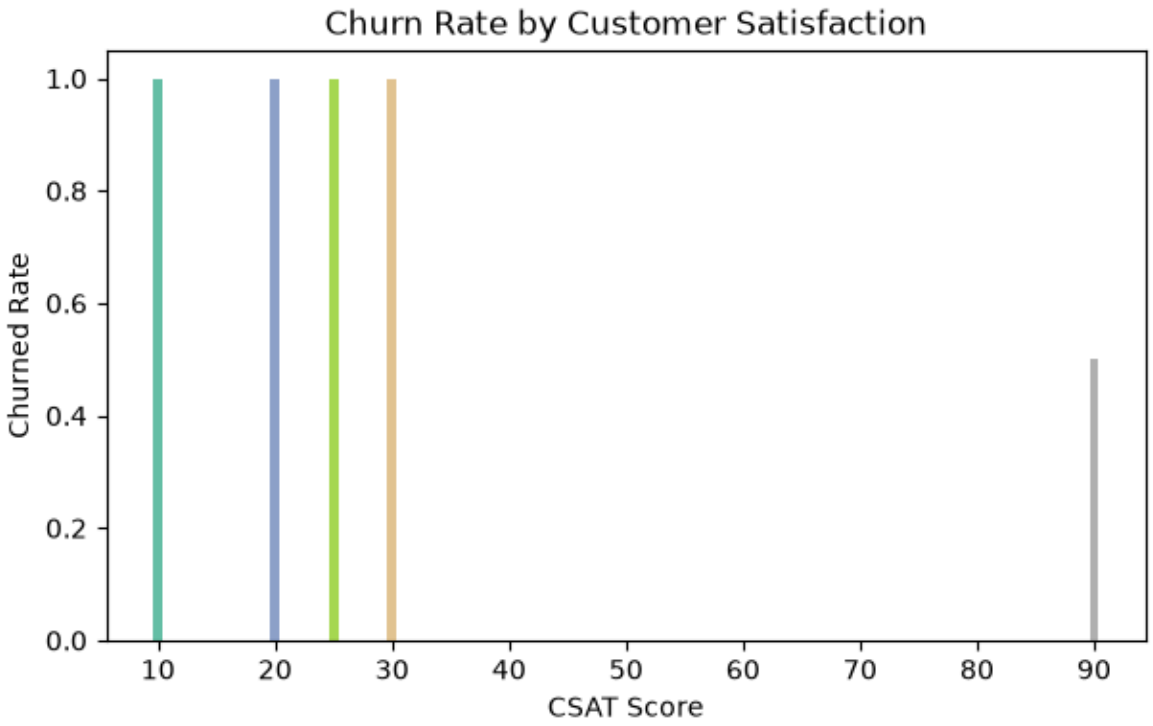
```
# 4.6 Churn by Customer Satisfaction
churn_plan = df_visual.groupby('csat_score')['churn_flag'].mean()

#colors = ['grey', 'gold', 'teal']
colors = plt.cm.Set2(np.linspace(0, 1, len(churn_plan)))
```

```
plt.figure(figsize=(7,4))

plt.bar(churn_plan.index, churn_plan.values, color = colors)

plt.title('Churn Rate by Customer Satisfaction')
plt.xlabel('CSAT Score')
plt.ylabel('Churned Rate')
plt.show()
```



In [ ]:
```
# Visulaization using Seaborn
```

In [167…
```
# encoding - convert str to numeric so that we can find corr between features
df_visual.columns
```

Out[167…
```
Index(['customerid', 'subscription_start_date', 'subscription_type',
       'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
       'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
       'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
       'complaint_date', 'escalations', 'csat_score', 'complaint_count',
       'escalation', 'churn_risk', 'cancellation_month'],
      dtype='str')
```

In [169…
```
# df_encoded =
df_visual[['plan_type', 'contract_type','churn_score',
       'churn_flag','churn_risk','escalation']].head()
```

Out[169...

	plan_type	contract_type	churn_score	churn_flag	churn_risk	escalation
0	Standard	Annual	12	0	low	0
1	Premium	Annual	91	1	high	1
2	Basic	Monthly	34	0	low	0
3	Premium	Annual	8	0	low	0
4	Standard	Monthly	88	1	high	1

In [174...

```
# incorrect method of encoding - as numbers are not assigned based on priority
df_encoded = df_visual[['plan_type', 'contract_type','churn_score', 'churn_flag'

categorical_cols = ['plan_type', 'contract_type', 'churn_risk']

for col in categorical_cols:
    df_encoded[col] = df_encoded[col].astype("category").cat.codes
```

In [175...

```
df_encoded.head()
```

Out[175...

	plan_type	contract_type	churn_score	churn_flag	churn_risk	escalation
0	2	0	12	0	1	0
1	1	0	91	1	0	1
2	0	1	34	0	1	0
3	1	0	8	0	1	0
4	2	1	88	1	0	1

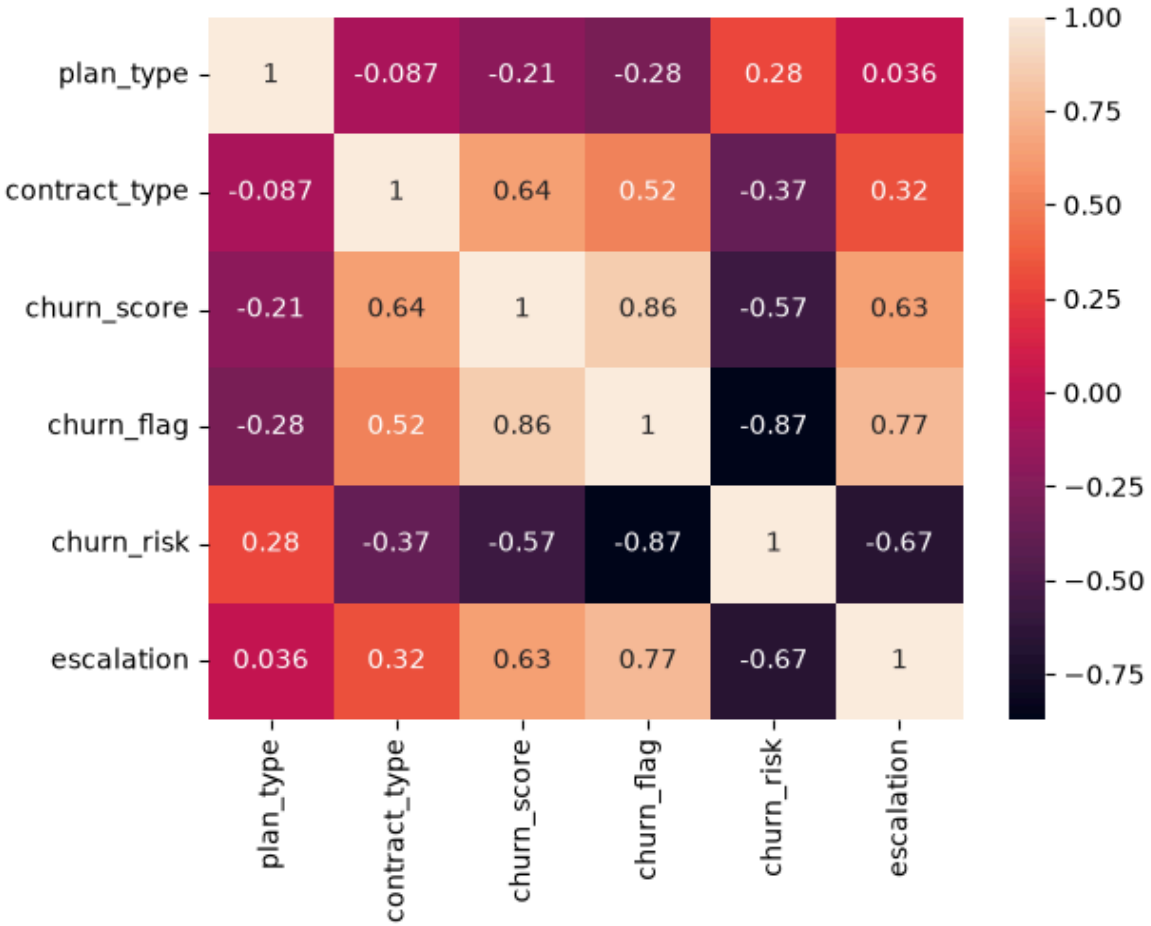
In [176...

```
# Heatmap (correlation matrix)

sns.heatmap(df_encoded.corr(), annot=True)
```

Out[176...

```
<Axes: >
```



In [179…
```
df_visual['churn_risk'].unique()
```

Out[179…
```
<StringArray>
['low', 'high', 'med']
Length: 3, dtype: str
```

In [182…
```
# correct method of encoding - based on priority
df_encoded = df_visual[['plan_type', 'contract_type','churn_score', 'churn_flag'

order_mappings = {
    'plan_type' : ['Basic','Standard', 'Premium'],
    'contract_type' : ['Monthly', 'Annual'],
    'churn_risk': ['low', 'high', 'med']
}
for col, order in order_mappings.items():
    df_encoded[col] = pd.Categorical(df_encoded[col].astype('category'), categor
```

In [184…
```
df_visual[['plan_type', 'contract_type','churn_score',
       'churn_flag','churn_risk','escalation']].head()
```

Out[184…

	plan_type	contract_type	churn_score	churn_flag	churn_risk	escalation
0	Standard	Annual	12	0	low	0
1	Premium	Annual	91	1	high	1
2	Basic	Monthly	34	0	low	0
3	Premium	Annual	8	0	low	0
4	Standard	Monthly	88	1	high	1

In [183…

```
df_encoded.head()
```

Out[183…

	plan_type	contract_type	churn_score	churn_flag	churn_risk	escalation
0	1	1	12	0	0	0
1	2	1	91	1	1	1
2	0	0	34	0	0	0
3	2	1	8	0	0	0
4	1	0	88	1	1	1

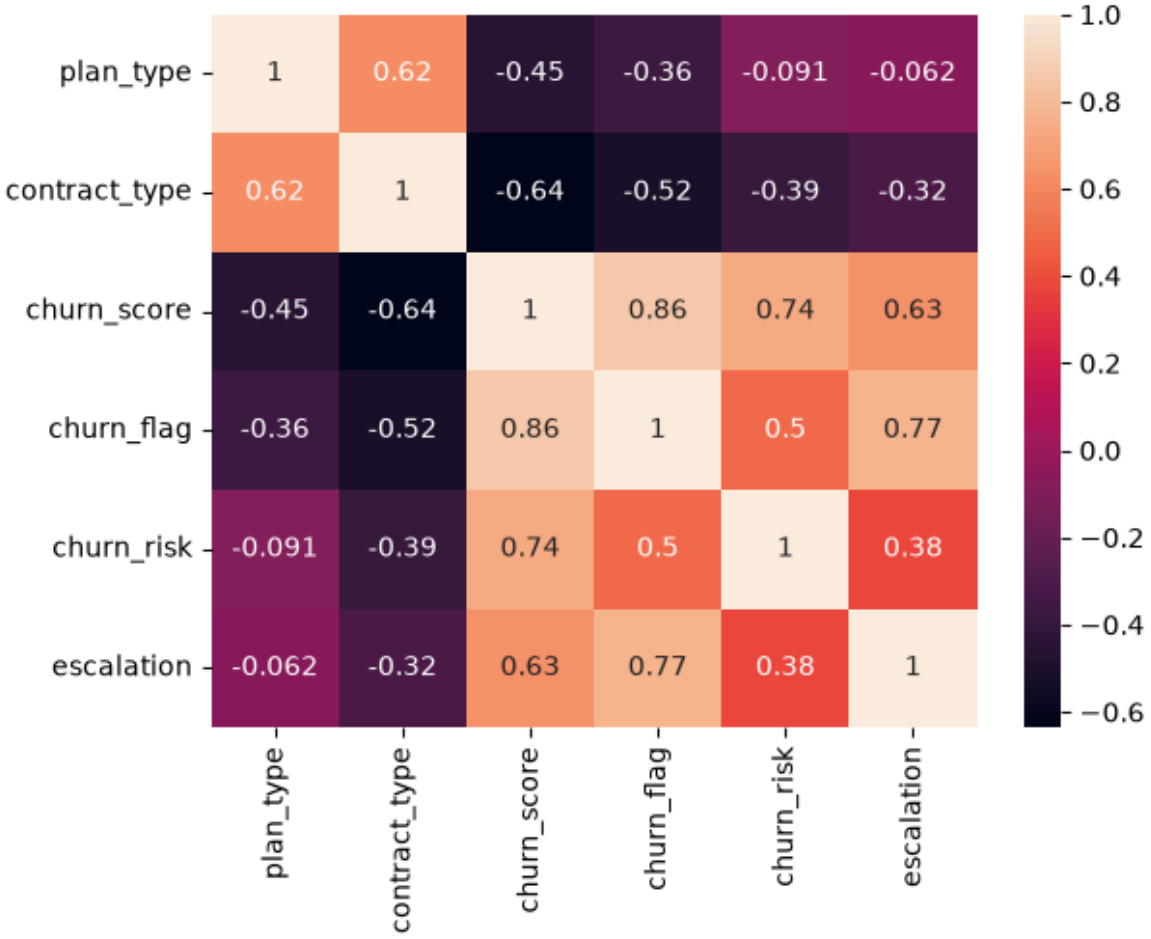
In [185…

```
# Heatmap (correlation matrix)

sns.heatmap(df_encoded.corr(), annot=True)
```

Out[185…

```
<Axes: >
```

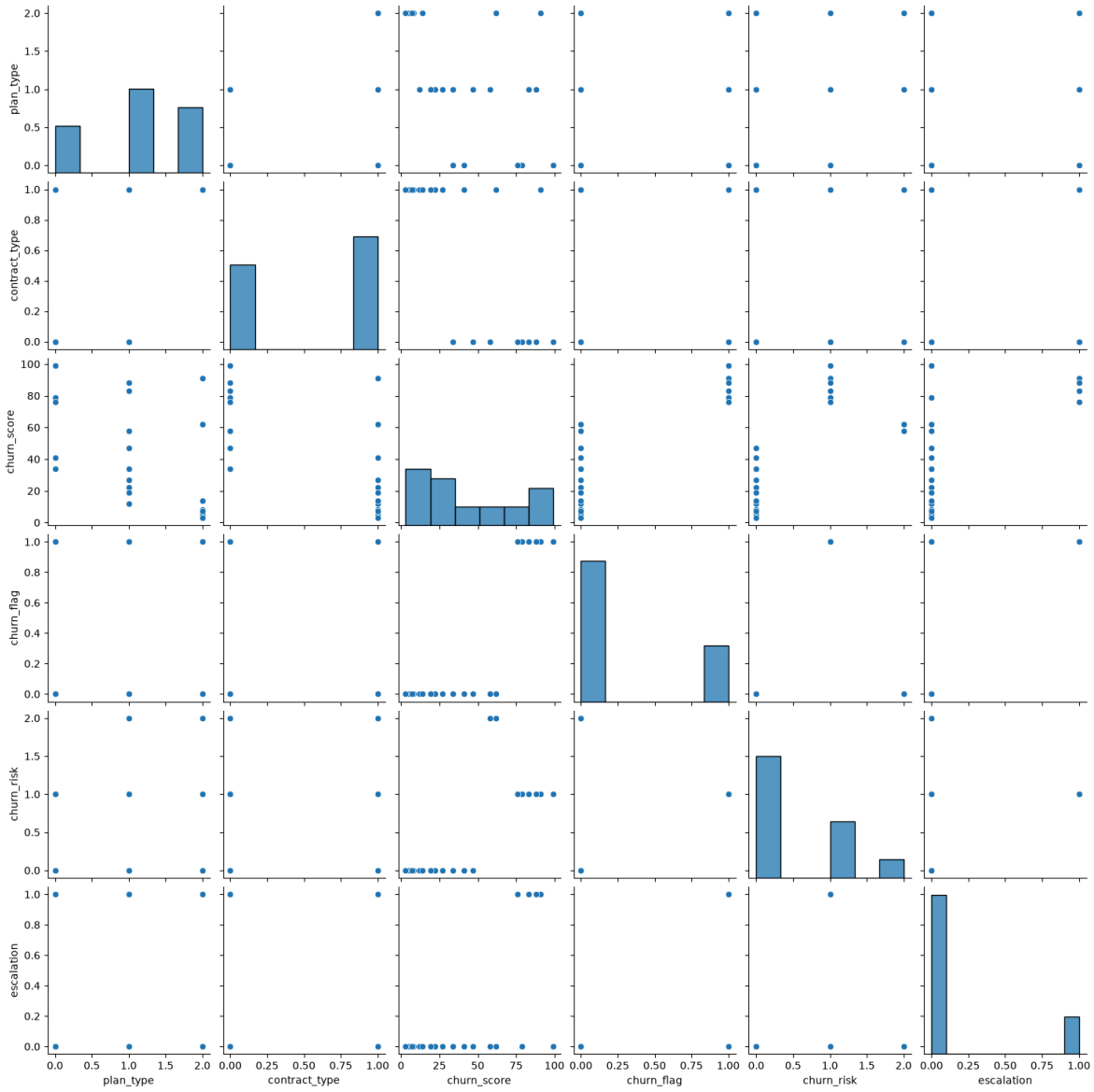


In [186…

```
# pairplot - relationship in a dataset
sns.pairplot(df_encoded)
```

Out[186…

```
<seaborn.axisgrid.PairGrid at 0x1e6d0868d70>
```



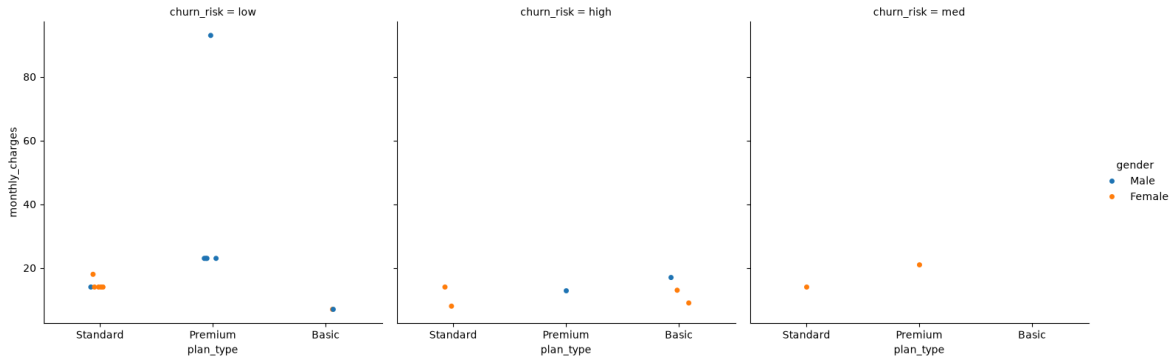
In [187…
```
df_visual.columns
```

Out[187…
```
Index(['customerid', 'subscription_start_date', 'subscription_type',
       'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
       'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
       'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
       'complaint_date', 'escalations', 'csat_score', 'complaint_count',
       'escalation', 'churn_risk', 'cancellation_month'],
      dtype='str')
```

In [188…
```
# catplot/Facegrid plot - multi-dim comparison

sns.catplot(data=df_visual,
            x='plan_type',
            y='monthly_charges',
            hue='gender',
            col='churn_risk')
```

Out[188…
```
<seaborn.axisgrid.FacetGrid at 0x1e6d322d400>
```



In [ ]:

In [ ]:
```
# pivot table
```

In [190…
```
pd.pivot_table(
    df_visual,
    index='plan_type',
    values='churn_flag',
    aggfunc = 'mean'
).reset_index()
```

Out[190…

	plan_type	churn_flag
0	Basic	0.600000
1	Premium	0.142857
2	Standard	0.222222

In [192…
```
pd.pivot_table(
    df_visual,
    index='plan_type',
    values=['monthly_charges', 'customerid', 'churn_flag'],
    aggfunc = {
        'monthly_charges' : 'sum',
        'customerid' : 'nunique',
        'churn_flag' : 'mean'
    }
).reset_index()
```

Out[192…

	plan_type	churn_flag	customerid	monthly_charges
0	Basic	0.600000	5	52.95
1	Premium	0.142857	7	218.93
2	Standard	0.222222	9	123.91